
Алгоритмы

Лекции

Конспект: Ivan Gerunov

03.09.2026

Содержание

1. Асимптотика алгоритмов	3
1.1 Зачем нужна асимптотика	3
1.2 О-нотация	3
1.3 Худший, средний и лучший случаи	4
1.4 Иерархия классов сложности	4
2. Сортировки: слияние и быстрая сортировка	5
2.1 Сортировка слиянием	5
2.2 Быстрая сортировка	5
2.3 Сравнение	6

1. Асимптотика алгоритмов

1.1 Зачем нужна асимптотика

Время работы алгоритма зависит от входа, поэтому его удобно описывать не точной формулой, а **скоростью роста** при $n \rightarrow \infty$, где n — размер входа. Это позволяет сравнивать алгоритмы независимо от конкретного железа и константных множителей.

1.2 О-нотация

Определение1 (О-большое)

Функция $f(n) = O(g(n))$, если существуют константы $c > 0$ и n_0 такие, что для всех $n \geq n_0$ выполняется

$$f(n) \leq c \cdot g(n).$$

Иными словами, $g(n)$ — асимптотическая верхняя оценка для $f(n)$.

Определение2 (Омега-большое)

Функция $f(n) = \Omega(g(n))$, если существуют константы $c > 0$ и n_0 такие, что для всех $n \geq n_0$ выполняется $f(n) \geq c \cdot g(n)$ — то есть $g(n)$ является асимптотической нижней оценкой.

Определение3 (Тета-большое)

$f(n) = \Theta(g(n))$, если одновременно $f(n) = O(g(n))$ и $f(n) = \Omega(g(n))$ — то есть $g(n)$ оценивает $f(n)$ и сверху, и снизу с точностью до константы.

Пример 1 — линейный поиск

Перебор всех n элементов массива в худшем случае делает n сравнений, значит время работы — $O(n)$.

Пример 2 — бинарный поиск

На каждом шаге бинарного поиска область поиска уменьшается вдвое, поэтому число шагов не превышает $\log_2 n + 1$ — время работы $O(\log n)$.

1.3 Худший, средний и лучший случай

Асимптотика без уточнения обычно означает **худший случай** (worst case) — верхнюю границу для любого входа заданного размера. Средний случай (average case) требует предположений о распределении входов, а лучший случай (best case) редко даёт полезную информацию о надёжности алгоритма.

1.4 Иерархия классов сложности

$$O(1) \subset O(\log n) \subset O(n) \subset O(n \log n) \subset O(n^2) \subset O(2^n)$$

Далее в курсе эти обозначения будут использоваться без отдельных напоминаний — при анализе каждого нового алгоритма нужно уметь быстро прикинуть, к какому из этих классов он относится.

2. Сортировки: слияние и быстрая сортировка

2.1 Сортировка слиянием

Определение4 (*merge sort*)

Рекурсивный алгоритм «разделяй и властвуй»: массив делится пополам, каждая половина сортируется рекурсивно, затем две отсортированные половины сливаются (**merge**) в один отсортированный массив за линейное время.

Слияние двух отсортированных массивов длины $n/2$ каждый делается за $O(n)$ — идём по обоим одновременно и на каждом шаге забираем меньший из текущих элементов. Рекуррентное соотношение для времени работы:

$$T(n) = 2T(n/2) + O(n), \quad T(1) = O(1).$$

Пример 3 — разбор рекурренты

По основной теореме о рекуррентных соотношениях (master theorem) при $a = 2$, $b = 2$, $f(n) = O(n)$ получаем $T(n) = O(n \log n)$ — и в худшем, и в среднем случае.

Сортировка слиянием **стабильна** (не меняет порядок равных элементов) и гарантирует $O(n \log n)$ даже в худшем случае — в отличие от быстрой сортировки. Плата за это — $O(n)$ дополнительной памяти на слияние.

2.2 Быстрая сортировка

Определение5 (*quicksort*)

Выбирается опорный элемент (**pivot**), массив разбивается на элементы меньше и больше опорного (партиционирование за $O(n)$), затем обе части сортируются рекурсивно.

Пример 4 — худший случай

Если опорным всегда выбирать первый элемент уже отсортированного массива, каждое разбиение даёт части размера 0 и $n - 1$ — рекурсия вырождается в $T(n) = T(n - 1) + O(n)$, что даёт $O(n^2)$.

Пример 5 — средний случай

При случайном выборе опорного элемента (или случайной перестановке входа) ожидаемое время работы — $O(n \log n)$: асимптотически такое же, как у сортировки слиянием, но с меньшей константой и без дополнительной памяти.

2.3 Сравнение

Алгоритм	Худший случай	Средний случай	Память
Сортировка слиянием	$O(n \log n)$	$O(n \log n)$	$O(n)$
Быстрая сортировка	$O(n^2)$	$O(n \log n)$	$O(\log n)$

На практике быструю сортировку почти всегда предпочитают сортировке слиянием благодаря работе на месте (in-place) и меньшим константам — если только не нужна гарантия $O(n \log n)$ в худшем случае или стабильность.